# BVS1 – Betriebssysteme

Lecture 4 – Where Our Code Runs · Winter Semester 2026/2027

Stavros Kousidis

TH Köln · ICCT

# Locate the Work

# Questions You Asked

- Where the hell am I?
- How do I edit the files I am supposed to edit?
- Where is the program counter (**pc**)?
- Where do I find constants in the disassembly?

CC BY-SA 4.0

# Revision

We can now create, run, pause, and continue one process.

We will revisit the pieces:

- **Where do we edit, build, and execute?**
- **Which bytes are instructions, and which are runtime data?**
- **What must the CPU preserve when execution is interrupted?**

Use the same `worker()` and `permute()` from **Exercise 2**.

CC BY-SA 4.0

# One System, Three Machine Roles

```
physical host: Linux / macOS / Windows
    |  hosts the VM; Multipass manages it
    v
Ubuntu VM bvs1: our development host
    |  runs editor, build tools, and QEMU
    v
QEMU emulates an RV64 target: CPU + RAM + UART
    |  target CPU executes RV64 instructions
    v
our program / growing BVS1 operating system
```

The Ubuntu VM is where we work. The RV64 machine is our target.

CC BY-SA 4.0

# Follow One Source Change

```
Ubuntu VM
    editor -> saved main.c
                   |
          cross compiler + linker
                   |
              RV64 ELF file
                   |
            QEMU loads it
                   v
RV64 target: instructions execute using target RAM
```

Make runs the build recipe. QEMU receives the linked ELF.

CC BY-SA 4.0

# Find the File You Are Editing

On the **physical host**:

```
multipass shell bvs1
```

Inside the **Ubuntu VM**:

```
cd ~/teaching-bvs1-code
pwd
ls
nano main.c
```

Where must you be when you run `make`?

# Edit, Save, Check

**nano**: edit → **Ctrl-O**, Enter → **Ctrl-X**.

**Vim**, if already familiar: **i** → edit → **Esc** → **:wq**.

Change one output label, then inspect the saved change:

```
git diff -- main.c
```

**Saving changes source code. It does not rebuild the executable.**

Handout: *Editing Source Files in Ubuntu — nano and Vim*.

# Rebuild, Run, Observe

Inside the **Ubuntu VM**:

```
make
make run
```

Compare the observed output with your prediction.

Exit QEMU: **Ctrl-a**, release, then **x**.

Suppose the old label is still printed. Which link in the chain would you check first?

**saved source → build input → successful build → executable → QEMU run**

CC BY-SA 4.0

# Trace One Execution

# Return to Exercise 2

```
static void worker(void)
{
    char value[] = "BVS1";
    permute(value);
    uart_puts("process: result: ");
    uart_puts(value);
    uart_puts("\n");
    process_exit(0);
}
```

`worker()` owns a **modifiable local array**, including its final `\0`.

Does calling **`permute()`** create another process?

# The Helper Pauses Before It Returns

```
static void permute(char value[])
{
    char tmp;
    tmp = value[0]; value[0] = value[1]; value[1] = tmp;
    /* print value before yield, as in Exercise 2 */
    process_yield();
    tmp = value[2]; value[2] = value[3]; value[3] = tmp;
}
```

The parameter `value` is a **pointer to the existing array**.

At yield, **neither `permute()` nor `worker()` has returned**.

# Reconstruct the Pause

Recall the output you produced in Exercise 2 to complete the table:

| Moment | Array contents | Active application calls |
|---|---|---|
| before `permute()` | `BVS1\0` | `worker()` |
| just before yield | ? | ? |
| after `permute()` returns | ? | ? |

Then explain:

**What must still exist while the OS is running between the two swaps?**

# Code Stays in `.text`

```
.text: instructions            stack: runtime data

worker code                    worker stack frame
permute code                   permute stack frame
process_yield code             yield-path call data

pc (program counter)           sp (stack pointer)
  -> current instruction          -> current stack position
```

A **stack frame** is the call data of one active call, not a copy of its instructions.

Did we forget **pc** in our context switch?

# sp and pc Are Different Kinds of CPU State

`sp` is the conventional ABI name for general-purpose register `x2`.

`pc` is different:

```
pc = program counter
   = address of the instruction being executed
```

The `pc` is **architectural CPU state**, but it is not one of `x0`–`x31`.

For our cooperative context switch:

```
saved ra → restored ra → ret  → execution continues there
```

**No saved context contains the live `pc`. It contains a continuation address: here `ra`, which `ret` turns back into `pc`.**

# Snapshot 1 — Before the Helper Call

```
.text                          process-owned stack storage
                               higher addresses
worker instructions            +--------------------------+
  initialize value             | worker call data         |
  call permute                 | value: B V S 1 \0        |
  ...                          +--------------------------+
                               lower addresses
```

`pc` selects a `worker()` instruction; `sp` selects a position in `p.stack`.

Which of these bytes will the permutation change?

# Snapshot 2 — `permute()` Is Active

```
.text                          process-owned stack storage
                               higher addresses
worker instructions            +--------------------------+
permute instructions           | worker: value B V S 1 \0 |
  swap first pair              +--------------------------+
  ...                          | permute call data        |
                               +--------------------------+
                               lower addresses
```

`pc` now selects a `permute()` instruction. The array still belongs to `worker()`.

The helper receives its address; it does not receive a new array.

# Snapshot 3 — After the First Swap

```
.text                        process-owned stack storage
                             higher addresses
same worker instructions     +--------------------------+
same permute instructions    | worker: value V B S 1 \0 |
  ...                        +--------------------------+
  call process_yield         | permute call data        |
                             +--------------------------+
                             lower addresses
```

The data changed. The instructions did not move onto the stack.

Next, `process_yield()` will transfer control to the OS.

# Snapshot 4 — The OS Is Running

```
CPU                          preserved process state

pc -> OS instruction         p.stack: suspended call data
sp -> OS call stack                   value V B S 1 \0
                             p.context:
                               saved sp -> p.stack
                               saved ra -> continuation
                               saved s0-s11 values
```

The process's stack memory stays in place while the CPU uses another context.

The saved `ra` is a **value pointing back into code**, not code itself.

# Snapshot 5 — Continue the Same Call

```
restore process context
          |
return through the suspended yield path
          |
same permute() call: second swap -> V B 1 S \0
          |
return to the same worker() call
```

Resume restores an execution. Calling `worker()` again would restart its work.

# Exit Is a Lifecycle Decision

Both yield and exit return control to the OS:

```
yield -> READY     -> may run again
exit  -> FINISHED  -> must not run again
```

The stack bytes might still exist after exit.

**Existing bytes do not make a finished process runnable.**

`process_run()` must enforce the lifecycle rule.

# Classify and Diagnose

# One Memory, Different Roles

| Part of target memory | Role |
|---|---|
| `.text` | machine instructions |
| `.rodata` | constant data, such as output strings |
| `.data` | stored initial values for static/global data |
| `.bss` | zero-initialized static/global data |
| reserved OS stack | runtime data of OS-side calls |
| process stack storage | runtime data of process-side calls |

A stack is a use of reserved memory, not automatically an ELF section.

CC BY-SA 4.0

# Which Bytes Are Constant?

```
uart_puts("process: result: ");
char value[] = "BVS1";
```

- The output label is constant data, normally in `.rodata`.
- `value` is a separate, modifiable local array.
- Its initialization does not make that array read-only.

Initial bytes may come from instructions or constant data; the array stays writable.

CC BY-SA 4.0

# Where Do `.data` and `.bss` Fit?

For static/global objects, typical placement is:

```
static int initialized = 7;  /* typically .data */
static int zeroed;           /* typically .bss */
```

The inspected Exercise 2 build has:

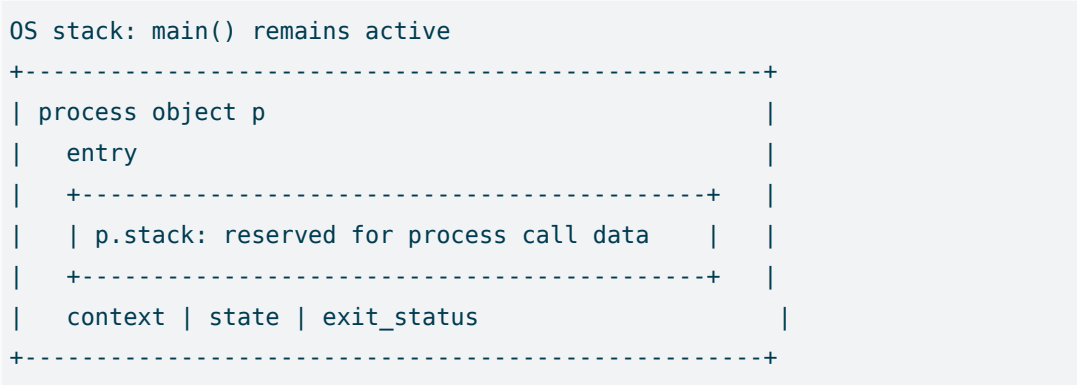
- `.bss`: saved OS context and current-process pointer;
- `.data`: empty.

The local value array is different again: it lives on the process stack while worker() is active.

CC BY-SA 4.0

# Where Is `p.stack` Actually Stored?

Exercise 2 declares `p` locally in `main()`:




Running the process selects a position inside `p.stack` using `sp`.

# Read the ELF, Then Read the Instructions

Inside the Ubuntu VM, after a successful build:

```
riscv64-unknown-elf-objdump -h build/hello-bvs1.elf
riscv64-unknown-elf-nm -n build/hello-bvs1.elf
riscv64-unknown-elf-objdump -d \
  --disassemble=context_switch build/hello-bvs1.elf
```

`-h`: sections · `nm -n`: symbols by address · `-d`: decode instructions.

Does a section/symbol listing show the live contents of `value`?

# What the Compiler May Change

The normal build uses optimization.

- `permute()` may be folded into `worker()` (**inlining**).
- A local value such as `tmp` may stay in a register.
- Initial array bytes may be supplied by instructions.

**A source-level call diagram is not an exact stack-frame layout.**

For the instructor's inspection build: `-O0 -g -fno-inline`.

CC BY-SA 4.0

# Diagnose the Broken Model

For each claim, identify the error and predict a consequence:

1. “Calling `permute()` starts another process.”
2. “The worker stack frame contains the worker instructions.”
3. “Saving `ra` is enough; the OS may reuse the process stack.”
4. “To resume, call the entry function again.”
5. “If its stack still exists, a FINISHED process may run.”

**Explain using code, storage, context, and lifecycle.**

CC BY-SA 4.0

# What If the Process Never Yields?

Imagine the process reaches this before its yield:

```
for (;;) { /* keep computing */ }
```

**Which part of our current mechanism gives the OS control back?**

An **interrupt** signals an event. When enabled, it transfers control to a configured OS handler.

A **timer interrupt** can enable **preemption**: taking control without a voluntary yield.

# Key Ideas

- **Development host:** where we edit, compile, link, and run QEMU.
- **Target:** where our RV64 instructions execute.
- **Code and call data:** instructions stay in `.text`; stack frames hold runtime data.
- **Storage and context:** stack bytes survive; saved registers reconnect execution to them.
- **Lifecycle:** READY permits another run; FINISHED does not.

Exercise 3: **locate → inspect → trace → diagnose** the same first process.

CC BY-SA 4.0

# References

Selected references:


- GNU Binutils: objdump and nm.
- GCC: Optimize Options.
- RISC-V International: ABIs Specification.
- Arpaci-Dusseau and Arpaci-Dusseau: *Operating Systems: Three Easy Pieces*.


CC BY-SA 4.0